

- [Hermetic Knowledge Isolation \(HKI\)](#)
  - [TL;DR](#)
  - [One-Picture Summary](#)
  - [Reader Guide](#)
  - [Executive Summary](#)
  - [Why Autonomy Makes Isolation Fundamental](#)
  - [HKI at a Glance](#)
  - [Glossary](#)
  - [The Problem HKI Solves](#)
  - [The Core HKI Thesis](#)
  - [System Model](#)
    - [A Worked Example: the Signed Scope Envelope](#)
  - [Architectural Consequences](#)
  - [Reference Architecture](#)
  - [Implementation Surface](#)
  - [Why HKI Matters for Agentic and MCP-Based Systems](#)
  - [Threat Model and Failure Modes](#)
    - [An Adversarial Walkthrough: Three Leak Paths, All Blocked](#)
  - [Novelty Assessment](#)
  - [Related Work](#)
  - [Technical Evaluation](#)
  - [Conformance and Regression Tests](#)
  - [Adoption and Migration Path](#)
  - [Theoretical Framing](#)
  - [Scientific Merit](#)
  - [Limits and Non-Claims](#)
  - [Anticipated Objections](#)
  - [Bottom Line](#)

# Hermetic Knowledge Isolation (HKI)

---

A Runtime Isolation Model for Enterprise Agentic Knowledge Systems

“

***Audience.** Enterprise AI, platform, security, and data architects building agentic RAG or MCP-style systems.*

***Reading time.** ~20 minutes for the full paper.*

## TL;DR

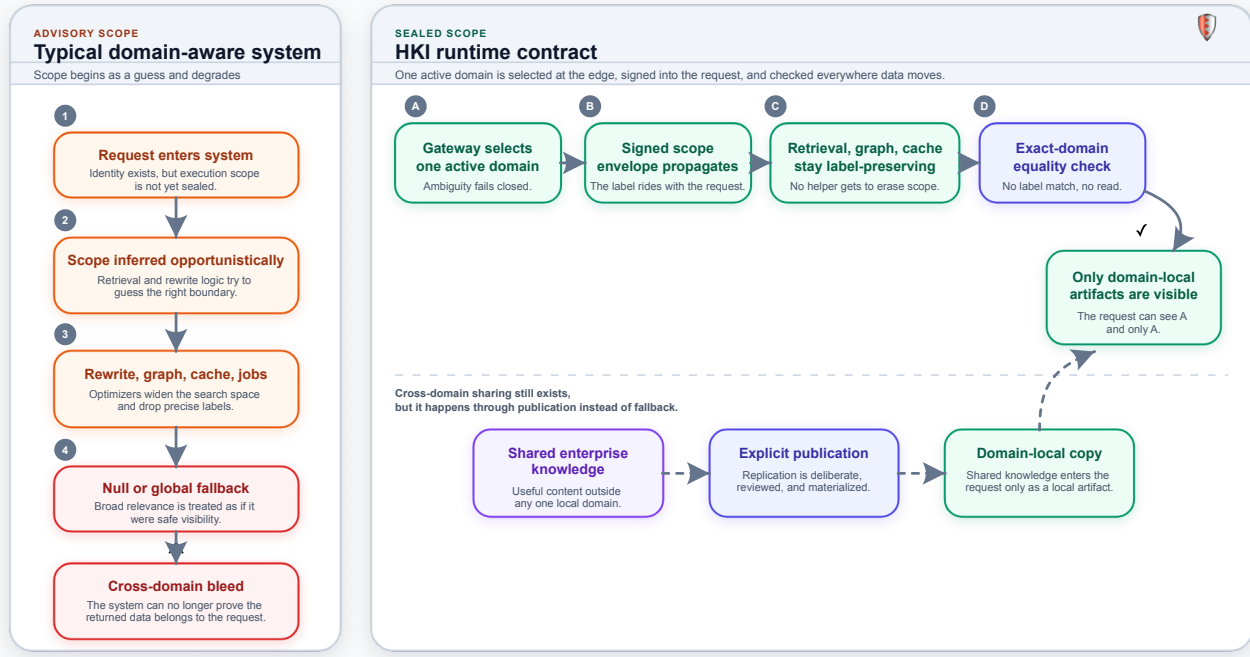
- 1 Most enterprise RAG and agent platforms claim to be “domain-aware,” but isolation often exists only as a final retrieval filter. Rewriters, graph edges, caches, jobs, traces, and review workflows can still lose or widen scope.
- 2 As agent autonomy increases, weak scope becomes an enterprise control failure. An autonomous agent that can rewrite, retrieve, remember, call tools, and start jobs must not be allowed to discover its boundary from convenience filters or natural language.
- 3 **Hermetic Knowledge Isolation (HKI)** turns isolation identity into a mandatory execution label: one domain per runtime artifact, one active domain per runtime operation, no null-scope, no global fallback, and no cross-domain visibility except through publication.
- 4 The primitives — labeled security, information-flow control, fail-closed authorization — are familiar. The contribution is a deployable whole-stack contract for agentic RAG and MCP-style systems, with conformance checks that can be audited before release.

## One-Picture Summary

The entire paper can be read as a contrast between two operating models. In the first, scope is inferred opportunistically and gradually dissolves as the request moves through rewriters, graph traversals, caches, and jobs. In the second, HKI pins one active domain at the edge, preserves that label across every transformation, and permits sharing only through explicit publication.

## The flow contrast at a glance

Conventional systems often weaken scope as they optimize.  
HKI preserves one active domain from the gateway to every downstream decision.



One-picture summary of HKI. The left side shows typical domain-aware RAG drifting from inferred scope toward null or global fallback and possible cross-domain bleed. The right side shows HKI selecting one active domain at the gateway, propagating a signed scope envelope, enforcing exact-domain equality, and limiting visibility to domain-local artifacts plus explicitly published copies.

## Reader Guide

If you are reading for different reasons, use the paper differently:

- 1 Core idea.** Read **TL;DR**, **One-Picture Summary**, **Executive Summary**, and **The Core HKI Thesis**.
- 2 How to build it.** Read **System Model**, **Threat Model and Failure Modes**, **A Worked Example**, and **Adoption and Migration Path**.
- 3 How to defend it.** Read **Novelty Assessment**, **Related Work**, **Theoretical Framing**, and **Scientific Merit**.
- 4 Tradeoffs and objections.** Read **Technical Evaluation**, **Limits and Non-Claims**, and **Anticipated Objections**.

The paper moves from plain-language explanation, to system design, to theory and evaluation. If you are a builder, the middle sections matter most.

# Executive Summary

---

Hermetic Knowledge Isolation, or HKI, is a runtime isolation model for enterprise knowledge and agent platforms. The basic idea is simple: every knowledge artifact belongs to one isolation domain, every runtime operation executes in one active domain, and shared knowledge appears in a domain only through explicit publication or replication. In data-sovereignty terms, HKI is an inference-time usage-control and provenance mechanism.

In plain language, HKI asks teams to stop treating scope as a search filter and start treating it as an execution label. That label must survive query rewriting, graph traversal, caching, ingestion, review, and tool calls. If the label is missing, ambiguous, or contradictory, the system fails closed.

That is why the word *hermetic* matters. A knowledge boundary is not a convenience filter, a UI hint, or a best-effort access rule. It is a sealed runtime boundary that governs what the system may observe and transform.

HKI does not claim to invent isolation. It builds on familiar ideas from information-flow control, mandatory access control, fail-closed authorization, and labeled systems. The contribution is operational: applying those ideas end to end to an agentic retrieval stack, where boundary failures often emerge from weakly scoped artifacts, graph edges, cache reuse, query rewriting, or background jobs rather than from one missing filter.

**Hermetic Value-Stream Isolation (HVSI)** is simply HKI expressed in value-stream language. When the isolation domain is a business value stream, HVSI and HKI are the same model.

The strongest claim for HKI is therefore modest but important: not that it invents isolation, but that it defines a precise, falsifiable, operationally enforceable isolation contract for enterprise agentic systems.

The practical test is blunt: a runtime path is HKI-conformant only if removing the active domain makes it fail, changing the active domain to an unauthorized value makes it fail, and adding artifacts in Domain B cannot change the observable output of a Domain A request except through explicit publication into A.

## Why Autonomy Makes Isolation Fundamental

---

Enterprise AI risk changes when systems move from answering questions to taking autonomous steps. A retrieval-only assistant may expose boundary mistakes through bad citations or overbroad search results. An agentic platform can do more: rewrite the request, choose tools, traverse graphs, consult memory, reuse caches, spawn async jobs, and trigger

follow-on workflows. Each of those capabilities creates a new place where scope can be lost, widened, cached, or reconstructed incorrectly.

That is why HKI treats isolation as a primitive construct. In an autonomous system, isolation cannot be a UI selection, a prompt instruction, a final SQL predicate, or a best-effort convention. It must be part of the execution object carried through the system. The platform must know, at every transformation, which domain the agent is operating in and which artifacts may legally shape the result.

For enterprises, the risk is not only direct data leakage. The larger risk is unauthorized influence: one domain's documents, graph edges, policy interpretations, cached answers, or tool results silently shaping another domain's autonomous decision path. This is especially dangerous in regulated functions, internal investigations, finance, pharmacy, legal, safety, supply chain, and customer operations, where a wrong boundary can change a decision without producing an obvious access violation.

HKI's position is therefore simple: autonomy requires isolation before optimization. Recall, memory, caching, tool use, graph traversal, and agent planning are valuable only after the system can prove the active domain is preserved. Without that proof, an enterprise agent is not merely under-secured; it is operating without a stable boundary for action.

## HKI at a Glance

---

| Concern                     | Typical “domain-aware” RAG             | Under HKI                                                    |
|-----------------------------|----------------------------------------|--------------------------------------------------------------|
| Artifact scope              | Optional metadata, often nullable      | Mandatory single domain label on every artifact              |
| Request scope               | Inferred per query, may be multi-scope | Exactly one active domain, signed at the gateway             |
| Agent autonomy              | Model/tool steps infer scope as needed | Tool, memory, job, and cache steps inherit the active domain |
| “Global” / shared knowledge | Null scope or wildcard fallback        | Explicit publication or replication into domain-local copies |
| Missing or ambiguous scope  | Falls back to broader visibility       | Fails closed                                                 |
| Enterprise risk             | Unauthorized influence is hard to see  | Boundary preservation is testable before release             |
| Graph traversal             | Inherits visibility from neighbors     | Domain label preserved on derived edges and nodes            |
| Cache keys                  | Query + model                          | Org + active domain + query + operation context              |
| Async jobs / connectors     | Reconstruct scope from inputs          | Carry signed domain through every stage transition           |
| Admin vs runtime queries    | Same code paths                        | Separate runtime plane and admin plane                       |
| Enforcement                 | Code review and convention             | Readiness audits, null-scope blocks, deploy-time gates       |

# Glossary

---

| Term                      | Meaning                                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Domain                    | A semantic isolation context (e.g., a value stream, business unit, regulated product line). The unit of hermetic execution.               |
| Active domain             | The single domain a runtime operation executes in, selected at the gateway or orchestrator, even if the principal is authorized for many. |
| Artifact                  | Any runtime object: document, chunk, graph node or edge, cache entry, review record, ingestion job, trace, derived output.                |
| $\sigma(a)$               | Domain label assigned to artifact <code>a</code> . Mandatory. Never null in the runtime plane.                                            |
| $\tau(r)$                 | Active domain label resolved for request <code>r</code> . Signed at the gateway. Propagated end to end.                                   |
| Signed scope envelope     | The tamper-resistant token that carries <code>(org, active_domain, authorized_domains, op, exp)</code> to every downstream service.       |
| Runtime plane             | The hermetic execution path: gateway, orchestrator, retrieval, ingestion, review, evaluators, tools, domain-scoped analytics.             |
| Admin plane               | A separate surface for cross-domain audit, reporting, and oversight. Never invoked from runtime code paths.                               |
| Publication / replication | The only authorized bridge between domains. Materializes new domain-labeled artifacts from a curated master.                              |
| Null-scope artifact       | An artifact with missing or empty domain identity. HKI treats this as a deploy-blocking defect, not a fallback.                           |
| HVSI                      | Hermetic Value-Stream Isolation — another name for HKI when the domain label is a business value stream.                                  |

## The Problem HKI Solves

Most enterprise RAG and agent platforms claim to be stream-aware, domain-aware, or tenant-scoped. In practice, many of them still permit subtle forms of cross-boundary leakage.

Said plainly: the system looks scoped at the UI or query layer, but scope quietly dissolves in the machinery underneath. The usual failure modes are not spectacular breaches; they are quiet architectural compromises:

- ◆ Null-scoped documents are treated as implicitly global.
- ◆ Wildcard semantics allow a scope such as `global` to mean *skip filtering*.
- ◆ Graph traversals move across boundaries through unlabeled derived nodes.
- ◆ Ingestion jobs, review records, and connector syncs do not carry first-class domain identity.
- ◆ Caches reuse retrieval results across logically distinct execution contexts.
- ◆ Corrective RAG or query rewriting broadens scope while trying to improve recall.
- ◆ Admin or migration paths leak into runtime code paths.

These are not merely implementation bugs. They arise because the system lacks a complete execution model for semantic isolation. HKI addresses that gap by making boundary identity a mandatory runtime property rather than an optional query parameter.

The enterprise version of this problem is especially severe because agentic systems often produce decisions, recommendations, workflow actions, and operational summaries rather than raw search results. A cross-domain artifact may never be shown to the user, but it can still influence what the agent decides to do next. HKI is designed to block that hidden influence path, not only visible document leakage.

## The Core HKI Thesis

HKI rests on four claims. Put plainly:

- 1 **Every artifact has one domain.** Documents, chunks, graph structures, review records, ingestion jobs, release artifacts, and scope-sensitive telemetry all belong to one isolation domain.
- 2 **Every runtime operation runs in one active domain.** A user may be authorized for many domains, and a business request may delegate across several of them, but each retrieval, tool, cache, memory, and synthesis step still executes in exactly one active domain.
- 3 **Shared knowledge is published, not globally visible.** If the same policy or playbook needs to appear in several domains, it is replicated into those domains explicitly.
- 4 **Ambiguity fails closed.** Missing scope, ambiguous scope, or unauthorized scope causes rejection, not fallback.

The formal statement is compact. For each artifact `a`, assign exactly one domain label `σ(a)` drawn from the set of domains `D`. For each runtime operation `r`, resolve exactly



one active domain  $\tau(r) \in D$ . Runtime visibility of artifact  $a$  to operation  $r$  is then permitted **only when all three conditions hold**:

- 1 The organization labels match.
- 2  $\tau(r) = \sigma(a)$  (exact-domain equality).
- 3 The relevant ACL policy authorizes the read or write.

If you remember only one rule from this paper, remember this one: **runtime visibility requires exact-domain equality**.

The forbidden family of rules is anything equivalent to:

“

*same org **and** (domain matches **or** domain is null **or** domain is global)*

HKI rejects null-domain fallback entirely. In a value-stream-oriented enterprise design, the domain label *is* the value stream; in that setting, HVSI is simply HKI.

## System Model

HKI becomes much easier to reason about when described as a small system model rather than as a slogan.

The moving parts are these:

- 1 **Principals**. Human users, agents, background jobs, connectors, and services.
- 2 **Domains**. The active isolation sets in which runtime execution is permitted.
- 3 **Artifacts**. Documents, chunks, graph nodes and edges, cache entries, review records, jobs, traces, and derived outputs.
- 4 **Requests**. Runtime executions that consume identity, authorization, and an active domain.
- 5 **Publication events**. Explicit transformations that create new artifacts for another domain without weakening runtime visibility rules.

The important point is that HKI is not only a read filter. It is a label-preserving execution discipline: once the active domain is chosen, every downstream transformation must preserve it unless the system is performing an explicit publication step.

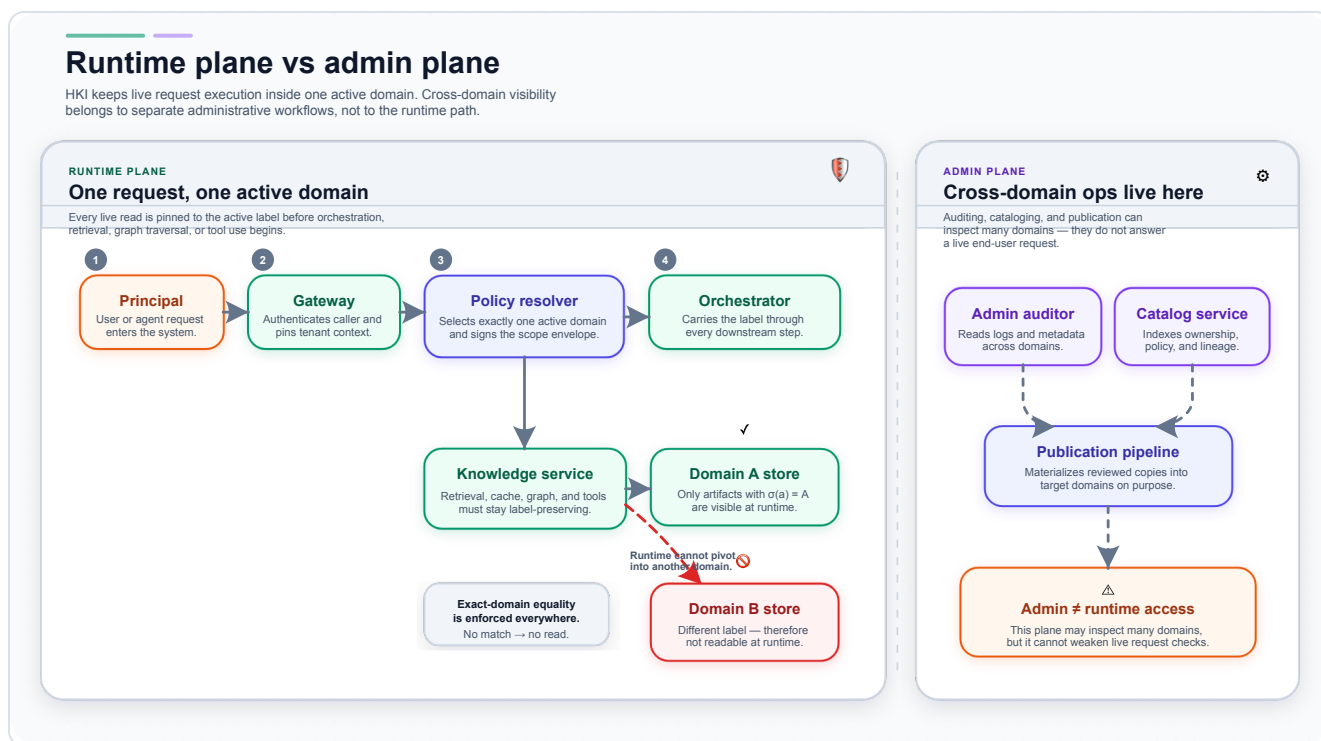
Each artifact has at least the following metadata, whether represented directly or implicitly through storage layout and signing rules:

- 1 Organization identity.
- 2 Domain identity.
- 3 Provenance or parentage.
- 4 Lifecycle state such as draft, reviewed, released, or archived.
- 5 Policy envelope controlling who may read, write, publish, or administer it.

Each runtime operation carries at least the following:

- 1 Authenticated principal identity.
- 2 Authorized domain set.
- 3 Exactly one selected active domain.
- 4 Operation type such as retrieve, ingest, review, publish, or administer.
- 5 Signed or otherwise tamper-resistant scope envelope propagated to downstream services.

The central operational rule is simple: **runtime transformations must preserve domain identity unless they are explicit publication steps**. A re-ranker does not widen scope. A graph traversal does not inherit neighboring visibility. A cache key does not erase the active domain. A connector does not ingest unlabeled content and hope filters will repair it later.



*Runtime plane vs admin plane: principals reach domain-scoped artifacts only through a policy resolver that pins exactly one active domain; the admin plane performs cross-domain inspection without weakening runtime filters.*

## A Worked Example: the Signed Scope Envelope

In practice, the gateway issues a short-lived envelope that downstream services must validate. A minimal shape is:

```
{
  "sub": "user:42",
  "org": "org_acme",
  "authorized_domains": ["vs_payments", "vs_fraud"],
  "active_domain": "vs_payments",
  "op": "retrieve",
  "iat": 1745000000,
  "exp": 1745000300,
  "sig": "..."
```

Downstream services apply three rules on every call:

- 1 **Verify** the signature and freshness.
- 2 **Reject** if `active_domain` is missing, null, `"global"`, or absent from `authorized_domains`.
- 3 **Bind** every storage read, cache key, graph traversal, tool invocation, and write to `(org, active_domain)` — never to `authorized_domains`.

A service that accepts the envelope but issues a query keyed only by `org` is not HKI-conformant, even if its results happen to look correct.

## Architectural Consequences

Two architectural consequences matter most.

First, HKI separates the platform into a runtime plane and an admin plane.

The runtime plane is hermetic. It includes retrieval, chat, ingestion, review, evaluation, release workflows, connector execution, and domain-scoped analytics that influence operator action. Every runtime API executes within one explicit domain.

The admin plane may inspect multiple domains, but only through dedicated admin-only queries, reports, dashboards, or audits. Cross-domain visibility is therefore a distinct privilege domain. It is not implemented by weakening runtime filters.

That distinction matters because many systems say they are isolated while reusing cross-domain operational queries inside live runtime endpoints. HKI forbids that pattern.

Second, HKI changes how shared knowledge works. If a policy or playbook must be visible in several domains, HKI requires publication into domain-local artifacts. That can be accomplished by a fan-out process, replication workflow, or controlled materialization pipeline, but not by leaving an artifact unscoped and relying on downstream code to treat it as enterprise-wide.

In short, reuse is realized by controlled derivation, not by shared runtime visibility.

## Reference Architecture

---

At a high level, the reference architecture has four jobs: choose the active domain, carry it, enforce it, and publish across domains only when explicitly authorized. In practice, HKI pushes enterprise agentic platforms toward a fairly specific architecture.

At the edge, a gateway or BFF authenticates the caller and resolves one active domain for the request. That decision is not advisory. It becomes part of the signed execution envelope. The orchestrator, retrieval services, ingestion services, review services, evaluators, and downstream tools must all consume and preserve that same domain label.

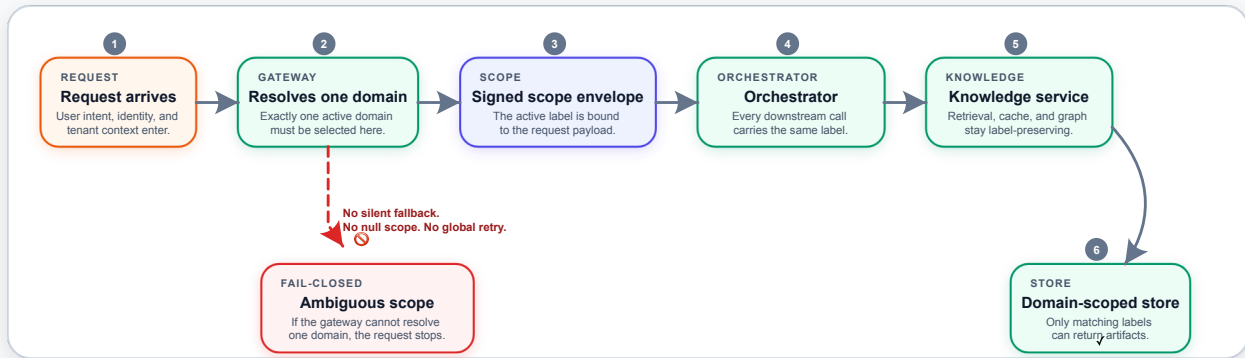
Within the runtime plane, every storage system is logically domain-aware even if it is physically shared. Vector indexes, graph stores, relational tables, object storage, caches, and job queues can be multi-tenant or multi-domain at the infrastructure level, but their runtime interfaces must enforce exact-domain semantics.

The result is a clean separation of concerns:

- 1 The gateway chooses and signs the active domain.
- 2 The runtime plane preserves that domain through every step.
- 3 The admin plane performs cross-domain audit and reporting without weakening runtime visibility.
- 4 Publication workflows are the only allowed bridges between domains.

## Request flow under HKI

The request becomes executable only after the system resolves one active domain. From that point on, the scope envelope is part of runtime, not an afterthought.

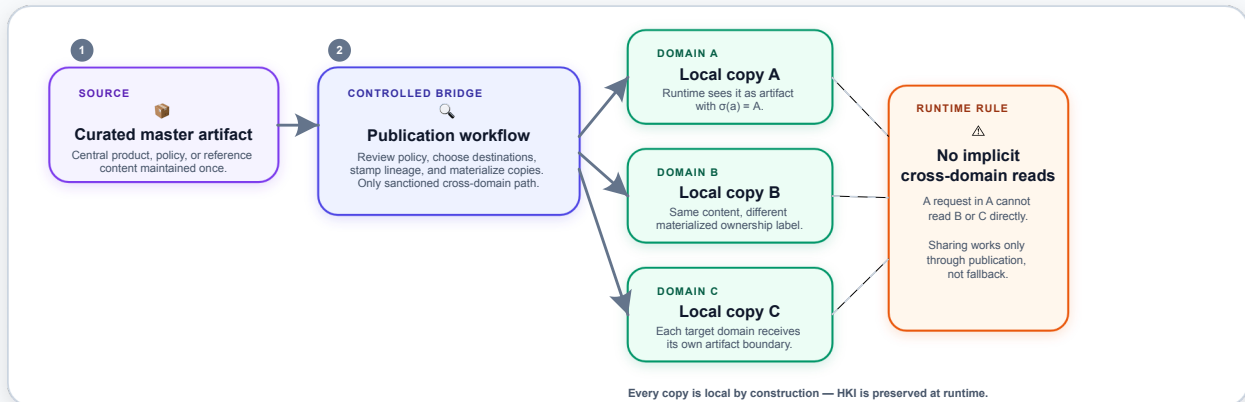


*Request flow under HKI: the gateway resolves exactly one active domain; ambiguous scope fails closed, otherwise a signed scope envelope is propagated through the orchestrator, knowledge service, and domain-scoped store.*

The publication path is intentionally different from the runtime path. Shared knowledge must move through explicit materialization.

## Explicit publication and fanout

HKI does not ban shared knowledge. It changes the mechanism: shared content becomes visible through reviewed materialization into each target domain.



*Publication path: a curated master artifact is materialized into per-domain copies through an explicit publication or replication workflow; domain copies have no implicit runtime visibility into one another.*

## Implementation Surface

HKI becomes credible only when every layer has a specific responsibility. The table below is a minimal implementation surface for an enterprise agentic platform.

| Layer                        | HKI Responsibility                                                                                     | Common Failure to Reject                                                                             |
|------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Gateway or BFF               | Resolve exactly one active domain, verify authorization, and issue a short-lived signed scope envelope | Defaulting to <code>global</code> , first authorized domain, or UI-selected scope without validation |
| Orchestrator and MCP gateway | Forward the envelope to every tool and prevent tool arguments from overriding runtime scope            | Letting a model or tool call infer a broader domain from natural language                            |
| Retrieval and vector store   | Bind every read, re-rank, and citation to <code>(org, active_domain)</code>                            | Applying a domain filter only to the final query, after rewrites or candidate expansion              |
| Graph store                  | Label nodes and edges, and require exact-domain traversal                                              | Following an unlabeled derived edge into another domain                                              |
| Ingestion and connectors     | Persist active domain on jobs, documents, chunks, extracted entities, and derived outputs              | Ingesting first and assigning scope later                                                            |
| Review and release workflows | Keep review records, approvals, evals, and release artifacts domain-labeled                            | Reusing cross-domain review queues inside runtime paths                                              |
| Caches                       | Include organization, active domain, operation, model/context version, and query fingerprint in keys   | Replaying a semantic cache hit from another domain                                                   |
| Admin plane                  | Provide separate audited cross-domain views that cannot be called by runtime endpoints                 | Sharing admin queries with live agent or retrieval paths                                             |
| Publication workflows        | Materialize new target-domain artifacts with provenance from a curated source                          | Treating a null or wildcard artifact as universally visible                                          |

A minimal conformance bar follows from that surface:

- 1 No runtime endpoint can answer without an explicit non-global active domain.
- 2 Every persisted runtime artifact class has a non-null domain identity.
- 3 Cache hits prove the same organization, active domain, operation, and context version.

- 4 Graph traversals reject unlabeled or different-domain nodes and edges.
- 5 Async jobs carry the signed domain, or a verified derivative, through every stage.
- 6 Publication creates new domain-labeled artifacts with provenance.
- 7 Readiness fails if null-scope audits or non-preserving paths are detected.

## Why HKI Matters for Agentic and MCP-Based Systems

HKI matters most where modern systems are most dynamic. Agents do more than issue direct document lookups. They compose tools, traverse graphs, reformulate queries, accumulate memory, and interact with caches, evaluators, and release workflows. The attack surface for boundary failure is therefore larger than in a conventional search application.

In an MCP-oriented architecture, HKI is not the bus — it is the policy law the bus must preserve.

If an enterprise MCP gateway is built under HKI, then every tool invocation must satisfy the following:

- 1 The auth envelope carries one explicit active domain.
- 2 Downstream tools cannot infer or broaden scope from user-provided arguments.
- 3 Mutating tools preserve domain ownership on every created artifact.
- 4 Cross-domain admin queries are separated from normal runtime tools.
- 5 Long-running workflows such as ingest, review, and release carry the same domain identity across all async stages.

This is what makes HKI more than a search filter. It is a boundary contract for the entire execution path.

## Threat Model and Failure Modes

HKI matters because modern agentic systems fail in more ways than conventional access-controlled applications. The primary risks are not only direct unauthorized reads. They also include semantic bleed through derived state, orchestration shortcuts, and operational convenience features.

The simplest way to read the table below is this: scope can fail because artifacts are weakly labeled, because execution steps widen or lose scope, or because operators reuse

convenience paths that were never meant for runtime traffic.

The threat model should therefore include both malicious and non-malicious failure paths:

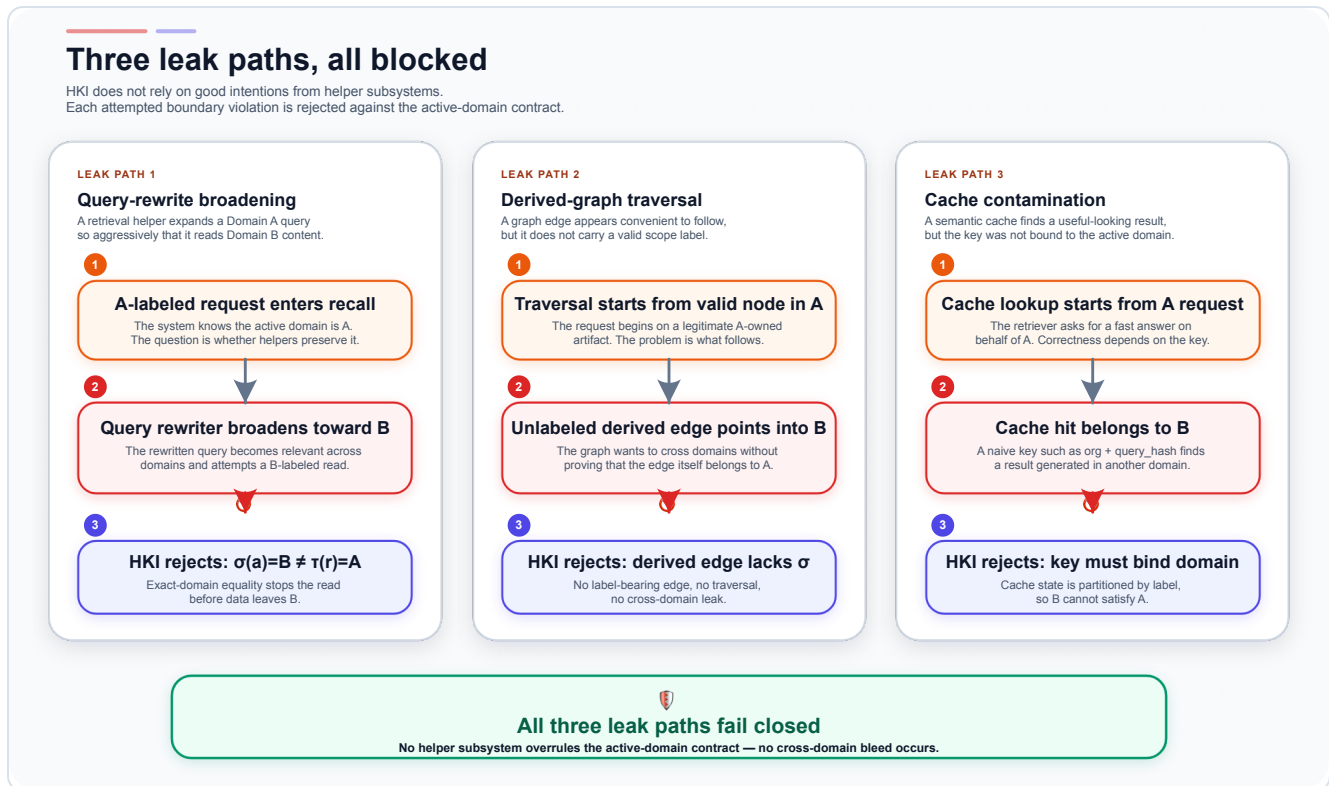
| Failure Path                                   | Typical Cause                           | Why It Happens                                                | HKI Control                                                                         |
|------------------------------------------------|-----------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Null-scoped artifact becomes globally visible  | Legacy migration or unlabeled ingest    | Scope is treated as optional metadata                         | Reject null-domain runtime semantics and audit continuously                         |
| Query rewriting broadens beyond caller context | Recall optimization                     | Retrieval stack treats relevance as more important than scope | Bind rewriting and retrieval to the signed active domain                            |
| Graph traversal crosses boundaries             | Derived edges lack labels               | Secondary structures inherit weak semantics                   | Require domain labels on derived graph artifacts and enforce exact-domain traversal |
| Cache contamination leaks results              | Cache keys omit domain                  | Optimization layer is built outside the security model        | Key caches by organization, domain, and operation context                           |
| Admin query reused in runtime path             | Code reuse under delivery pressure      | Operational tooling quietly weakens production guarantees     | Separate runtime plane and admin plane APIs                                         |
| Async job changes domain accidentally          | Worker reconstructs context incorrectly | Background systems treat scope as soft state                  | Persist and verify the active domain through every stage transition                 |
| Shared policy content bypasses publication     | Convenience shortcut                    | Teams want one copy visible everywhere                        | Use explicit publish or replicate workflows only                                    |

This is why HKI should be evaluated not only as an authorization scheme, but as a whole-system integrity discipline for knowledge movement.

### An Adversarial Walkthrough: Three Leak Paths, All Blocked



Consider a principal authorized in both Domain A and Domain B, issuing a request with `active_domain = A`. Three of the most common agentic leak paths are shown below, alongside the HKI check that rejects each.



Three common cross-domain leak paths under agentic RAG — query-rewrite broadening, derived-graph traversal, and cache contamination — each rejected by the HKI runtime check. The green note marks the fail-closed outcome.

- 1 **Query-rewrite broadening.** A corrective-RAG rewriter expands the query for recall. The retriever, keyed only by `org`, is about to read an artifact labeled  $\sigma(a)=B$ . HKI rejects the read because  $\tau(r)=A \neq \sigma(a)=B$ . The rewriter's broadened semantics never reach the store.
- 2 **Derived graph traversal.** GraphRAG follows neighbors of an A-labeled node. A derived edge — created by an offline extraction job that forgot to carry the label — points into a B-labeled node. HKI rejects the traversal: derived edges without  $\sigma$  fail closed, not open.
- 3 **Cache contamination.** The semantic cache was keyed by `(org, query_hash)` instead of `(org, active_domain, query_hash)`. A hit would return a B-labeled result. HKI rejects the read because the cache key does not bind the active domain.

The point is structural: each leak path is blocked by the *same* rule ( $\tau(r) = \sigma(a)$ ), applied at a *different* transformation. That is what “end-to-end label preservation” means in practice.

## Novelty Assessment

---

Novelty should be judged at the right layer.

At the primitive level, HKI is not highly novel. The underlying concepts are well known:

- 1 Labeled security and mandatory access control.
- 2 Fail-closed authorization.
- 3 Information-flow discipline and noninterference.
- 4 Explicit publication or replication workflows.
- 5 Scope-aware caching and policy propagation.

If presented as an invention of isolation itself, the claim would be overstated.

At the systems level, however, HKI is meaningfully novel. What is unusual is the insistence that the same semantic boundary be preserved end to end across:

- 1 BFF or gateway authentication.
- 2 Downstream JWT propagation.
- 3 Retrieval and reranking.
- 4 Graph traversal.
- 5 Ingestion and reprocessing.
- 6 Review and publication.
- 7 Evaluation and readiness gates.
- 8 Caches, traces, and other runtime artifacts.

Many enterprise AI systems stop at boundary-aware search. HKI is stronger because it treats isolation identity as a system invariant rather than as a local query condition.

The most distinctive combination in HKI is this:

- 1 One active domain per runtime execution, even if the user has broader authorization.
- 2 No null-domain runtime semantics.
- 3 Enterprise-wide knowledge delivered only through explicit publication or replication.
- 4 Operational enforcement through readiness audits and fail-closed health behavior.

That combination is not mathematically revolutionary, but it is sufficiently sharp and uncommon to be interesting as a serious enterprise systems contribution.

The most defensible summary is therefore this: *primitive-level novelty is low by design — HKI stands on decades of information-flow and labeled-security work. The systems contribution*

is meaningful because HKI names a falsifiable, end-to-end runtime isolation contract for agentic RAG and MCP-style execution fabrics.

## Related Work

---

This section answers a simple question: what is HKI borrowing, and what is it adding? Readers focused on implementation can skim it. The most relevant prior art falls in four clusters.

**Labeled security and information-flow control.** Bell and LaPadula’s lattice model [1](#) and Denning’s information-flow framework [2](#) established that confidentiality can be enforced by labels that flow with data through transformations. Decentralized Information Flow Control (DIFC) systems — Asbestos [3](#), HiStar [4](#), and Flume [5](#) — made this tractable at the OS and process level. Language-level descendants such as Jif [6](#) and IFDB [7](#) extended the idea to programs and databases. HKI inherits the invariant “labels flow with data” from this lineage but drops the requirement of a single trusted runtime, relocating enforcement to per-service checks under a signed envelope.

**Enterprise authorization at scale.** Google Zanzibar [8](#) defines the dominant industrial pattern for *who can access what*. Cedar [9](#) and OpenFGA [10](#) provide modern policy languages on top of similar substrates. HKI is complementary: Zanzibar-style systems answer *is this principal authorized for this resource?*, while HKI answers *which of the principal’s authorized domains is active for this request, and has that label been preserved by every transformation downstream?* HKI expects to be layered *above* Zanzibar-class engines, not to replace them.

**Tenant and row-level isolation in data platforms.** Row-level security in PostgreSQL, BigQuery row access policies, Snowflake row access policies, Pinecone namespaces, Weaviate multi-tenancy, Vertex AI Search ACL filtering, and Amazon Bedrock Knowledge Bases metadata filters all provide per-tenant or per-partition visibility. These mechanisms are necessary but not sufficient under HKI: they govern the final read, not the graph traversal, cache key, query rewrite, or async job that preceded it.

**LLM-era security guidance.** The OWASP Top 10 for LLM Applications [11](#) enumerates failure categories such as sensitive information disclosure (LLM06) and excessive agency (LLM08). NIST AI RMF [12](#) and the EU AI Act [13](#) frame high-risk AI systems as subjects of structural controls. HKI offers a concrete structural answer compatible with these frameworks: a runtime contract that narrows what an agentic system can observe, not merely what it is supposed to.

HKI is easiest to position by comparing it with adjacent controls:

| Control                     | What It Answers                                 | What HKI Adds                                                                                           |
|-----------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Tenant isolation            | Which customer or organization owns the data?   | One active semantic domain inside an enterprise tenant for each runtime execution                       |
| Row-level security          | Which rows can this principal read?             | Label preservation across rewrites, graph traversal, caches, jobs, tools, and derived artifacts         |
| Zanzibar, Cedar, or OpenFGA | Is this principal authorized for this resource? | Which authorized domain is active now, and whether every downstream transformation preserved it         |
| Vector metadata filters     | Which chunks should retrieval return?           | A whole-runtime rule that also covers candidate generation, re-ranking, citations, and cache population |
| Physical sharding           | Which database or cluster stores this data?     | A logical isolation contract that can run on shared infrastructure or isolated infrastructure           |
| Prompt and DLP filters      | Should the model output be blocked or redacted? | Prevention by narrowing what the agent can observe before generation                                    |

# Technical Evaluation

In engineering terms, HKI is strong because it is precise, testable, and operationalizable.

Its main strengths are:

- 1 **Fail-closed behavior.** Missing, ambiguous, or unauthorized domain context causes rejection rather than fallback.
- 2 **Compositionality.** The same domain label governs retrieval, ingest, graph, review, cache, and release behavior.
- 3 **Operational enforceability.** Null-domain audits and readiness blocks convert a design rule into a deploy-time gate.
- 4 **Shared infrastructure compatibility.** HKI does not require one database, cluster, or deployment per domain.

- 5 **Alignment with enterprise operating models.** Human authorization can be multi-domain while request execution remains singular and unambiguous.

Its main costs are also real. HKI buys clarity and boundary integrity, but it does so by making scope explicit everywhere:

- 1 Publication and replication increase workflow complexity.
- 2 Storage can grow when the same content is materialized into several domains.
- 3 Cross-domain analytics become an explicit admin-plane problem rather than a free side effect.
- 4 User experience becomes stricter because ambiguity is rejected.
- 5 Migration is substantial because legacy null-scoped artifacts must be assigned, replicated, or removed.

The most important technical claim is not just that filtered search results are correct. It is that every transformation is domain-preserving unless it is an explicit publication step. That includes reprocess, refresh, sync, review, chunking, graph extraction, evaluation, and cache population. This turns the whole pipeline into part of the security boundary.

## Conformance and Regression Tests

HKI should be tested as a boundary invariant, not as a documentation promise. A useful regression suite includes at least the following cases.

| Test                       | Failure Being Exercised                                          | Expected HKI Behavior                                             |
|----------------------------|------------------------------------------------------------------|-------------------------------------------------------------------|
| Missing active domain      | Runtime endpoint receives no domain context                      | Reject before retrieval, tool execution, or job creation          |
| Unauthorized active domain | Principal is authorized for A but request asks for B             | Reject at the gateway or first policy check                       |
| Null or global artifact    | Runtime store contains unlabeled or wildcard-labeled data        | Block readiness or make artifact invisible to runtime paths       |
| Query rewrite broadening   | Corrective RAG expands the query toward another domain           | Preserve the original active domain through retrieval             |
| Cross-domain graph edge    | Graph traversal encounters an unlabeled or B-labeled edge from A | Reject the edge or stop traversal                                 |
| Cache key missing domain   | Cache lookup is keyed by organization and query only             | Miss the cache or reject the read as non-conformant               |
| Async job loses scope      | Worker reconstructs scope from payload or defaults               | Fail the stage transition before producing artifacts              |
| Runtime calls admin query  | Live request attempts to use cross-domain operational query      | Deny the call and log a plane-separation violation                |
| Publication fan-out        | Shared policy is published from a curated source into A and B    | Create separate A-labeled and B-labeled artifacts with provenance |

Useful acceptance thresholds are concrete: zero successful cross-domain reads in the adversarial suite, zero runtime endpoints that answer without active domain, zero cache keys that omit active domain, zero async job records without domain identity, and a readiness block whenever null-scope runtime artifacts are detected.

## Adoption and Migration Path

For practitioners, this is the most operational section of the paper. Most enterprises cannot adopt HKI in one cutover. They have existing corpora, weakly labeled jobs, mixed caches,

and operational tools that assume broad visibility. A credible rollout therefore needs phases.

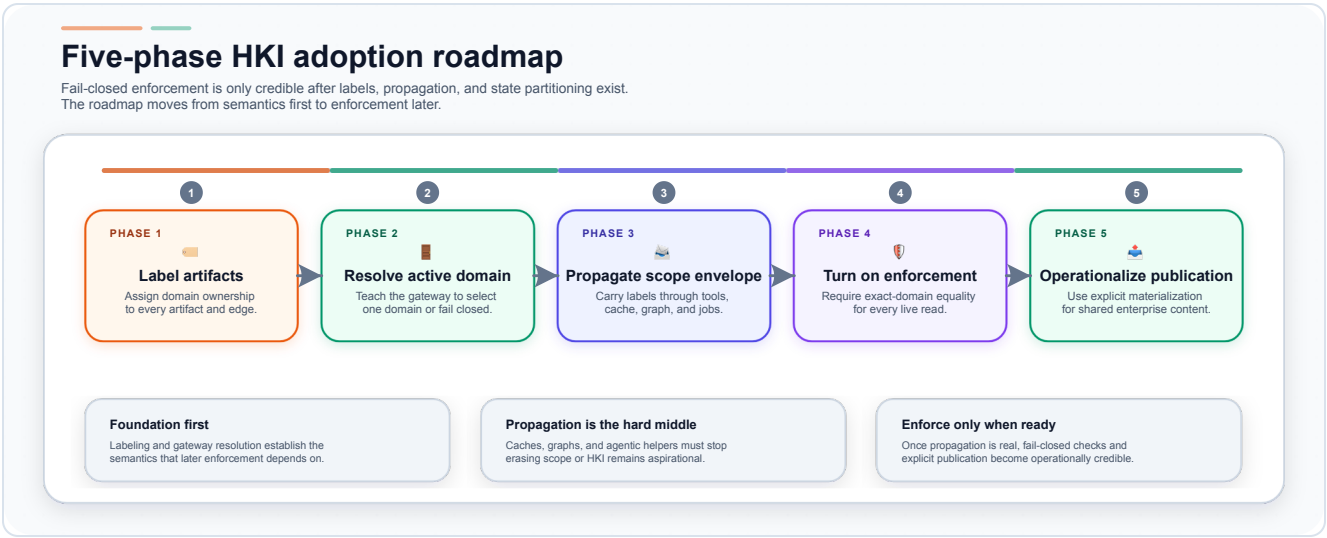
Phase 1 is inventory and audit. Count null-scoped artifacts, unlabeled derived structures, cache keys that ignore domain, and runtime endpoints that can answer without an explicit domain.

Phase 2 is label normalization. Every artifact class receives one required domain identity. If content is genuinely shared, define a publication source and destination model rather than permitting an unlabeled global bucket.

Phase 3 is propagation hardening. The gateway resolves one active domain, signs it, and all downstream services reject missing or contradictory context. This is where most hidden coupling becomes visible.

Phase 4 is runtime fail-closed enforcement. Readiness checks, audits, and release gates block deployment when null-domain artifacts or non-preserving paths remain.

Phase 5 is operational maturation. Add domain-aware observability, adversarial regression tests, publication workflows, and periodic audit reports that prove the boundary still holds as the platform evolves.



*Five-phase HKI adoption roadmap. Each phase builds on the previous one; fail-closed enforcement (Phase 4) is only credible once labels and propagation are in place.*

The migration cost is real, but the payoff is clear: domain isolation moves from convention to contract.

# Theoretical Framing

Readers focused on shipping systems can skim this section. Theoretically, HKI is best viewed as an applied noninterference model for enterprise agentic knowledge systems.

Put more simply: changes in Domain B should not change what a Domain A request can see unless the system explicitly published that content into A.

Informally, the intended property is:

For any two domains `d1` and `d2`, where `d1 != d2`, changes to artifacts labeled `d2` should not affect the observable outputs of runtime executions labeled `d1`, except through explicitly authorized publication that materializes new artifacts labeled `d1`.

That is recognizably a noninterference-style statement. It is not a new branch of theory, but it is a rigorous specialization of known theory to a modern execution environment where the observable output includes:

- 1 Retrieval results.
- 2 Graph traversals.
- 3 Generated answers.
- 4 Citations.
- 5 Cache hits and misses.
- 6 Review and release state.
- 7 Derived analytics that alter operator action.

The theoretical strengths of HKI are:

- 1 It distinguishes principal authorization from runtime observation scope.
- 2 It treats execution context as singular rather than set-valued.
- 3 It models shared knowledge as controlled derivation, not default visibility.
- 4 It supports label-preservation reasoning across transformations.

The theoretical limits are equally clear:

- 1 It does not introduce a new formal logic or type system.
- 2 It depends on known ideas from information-flow control and labeled security.
- 3 Its strongest originality is architectural and systems-theoretic rather than foundational.

A useful theorem sketch is the following.



“

**Theorem (sketch) — Runtime Noninterference under HKI.** *If every runtime artifact has exactly one domain label, every runtime operation has exactly one active domain label, every runtime read enforces exact domain equality, every runtime write preserves or explicitly assigns the current domain label, caches are keyed by domain, and cross-domain reuse occurs only through explicit publication that materializes new domain-labeled artifacts, then runtime outputs satisfy a noninterference-style isolation property, modulo side channels and shared-model effects.*

That is a respectable theoretical contribution for a systems paper even if it is not a new theory paper.

## Scientific Merit

If HKI is framed as a paper rather than only as an engineering doctrine, it is strongest as a systems-security contribution. The right question is not “did HKI invent labels?” The right question is whether HKI eliminates leak paths with acceptable cost.

A credible scientific framing would be:

- 1 **Problem:** enterprise agentic retrieval systems often leak across semantic boundaries because isolation is enforced only at query time while artifacts, graph structure, jobs, caches, and release workflows remain weakly labeled.
- 2 **Contribution:** HKI, an end-to-end runtime isolation model with hermetic domain execution and explicit publication-based sharing.
- 3 **Method:** implement stream-preserving execution across a multi-service agentic platform and evaluate isolation behavior under adversarial tests.
- 4 **Result:** demonstrate elimination of cross-domain bleed with bounded latency, operational overhead, and storage amplification.

The right evaluation agenda would include:

- 1 Cross-domain leak rate under adversarial retrieval and graph queries.
- 2 Null-scoped artifact count over time.
- 3 Unauthorized request rejection accuracy.
- 4 Cache contamination rate across domains.

- 5 Graph leakage rate through derived structures.
- 6 Latency overhead of strict domain binding.
- 7 Storage amplification under publication and replication.
- 8 Operational burden of migration and audit enforcement.

The right baselines would be:

- 1 Org-wide retrieval with optional domain filters.
- 2 Domain filters with null or global fallback.
- 3 Row-level filtering without publication-plane separation.
- 4 Domain-aware retrieval without domain-aware cache, graph, or workflow enforcement.

Under that framing, HKI has real scientific value. The novelty is not that it invents security labels. The novelty is that it defines and enforces a whole-system isolation contract for agentic enterprise knowledge.

## Limits and Non-Claims

---

HKI should not be described as solving all enterprise isolation problems. Its claim is narrower: strong runtime isolation of knowledge artifacts and transformations, not total security.

It does not by itself address:

- 1 Model-side memorization or provider-side leakage.
- 2 Side channels such as timing, cost, or traffic analysis.
- 3 Governance questions about who should be allowed into which stream.
- 4 The economics of replication at very large scale.

It also does not require physical sharding. One cluster, one database, or one vector store may still host many domains so long as the runtime isolation invariants are preserved.

That is why HKI is best described as a runtime isolation guarantee, not an infrastructure topology.

## Anticipated Objections

---

These are the most likely skeptical questions from reviewers and practitioners.

**“Isn’t this just row-level security with extra steps?”** Row-level security is one mechanism HKI may use at the storage layer, but HKI is a whole-system contract. Row filters do not by themselves govern graph traversal, cache keys, async job context, query rewriting, or release workflows. HKI requires that every transformation in the runtime plane preserve the domain label, not only the final read.

**“We already filter by tenant. How is domain different?”** Tenant isolation typically separates customers of a multi-tenant SaaS. Domain isolation under HKI separates *semantic execution contexts* inside a single enterprise — for example, value streams, business units, or regulated product lines that share infrastructure but must not share runtime visibility. A user may be authorized in many domains; a request still executes in exactly one.

**“Won’t publication blow up our storage?”** Replication amplifies storage for genuinely shared content. In practice the shared corpus is small relative to domain-local content, and the cost is bounded and predictable. The alternative — implicit global visibility — trades unbounded leakage risk for marginal storage savings.

**“Doesn’t fail-closed behavior hurt UX?”** It changes UX. Ambiguous requests must be disambiguated rather than silently broadened. In agentic systems this is usually desirable: the model gets a narrower, more trustworthy context and the operator gets a clear signal when scope is missing.

**“Is HKI specific to one vendor or stack?”** No. HKI is a runtime contract. It can be implemented over Postgres + pgvector, AlloyDB, BigQuery, OpenSearch, a graph store, or an MCP gateway. What matters is that the active domain is signed at the edge and preserved by every downstream service.

**“What about the LLM itself memorizing across domains?”** Out of scope. HKI governs the platform’s runtime artifacts and execution paths. Model-side memorization, provider-side leakage, and side channels (timing, cost, traffic) require separate controls.

## Bottom Line

---

HKI is not a new foundational security primitive. It is a strong and timely application of labeled isolation and noninterference principles to the specific realities of enterprise agentic knowledge systems.

Its most defensible contribution is this:

HKI turns isolation identity into a mandatory execution label for agentic retrieval and knowledge workflows, eliminates null-domain runtime semantics, separates runtime hermeticity from admin-plane visibility, and requires shared knowledge to be delivered by publication rather than fallback visibility.

That is a strong technical idea, a useful theoretical specialization, and a meaningful systems contribution.

---

*Feedback, critique, and counter-examples are welcome. The model is intentionally falsifiable: the sharpest critique is a concrete enterprise deployment in which null-domain runtime semantics are both safe and necessary.*

---

Henok Ghebrechristos, 2026.

---

- 1 Bell, D. E., LaPadula, L. J. *Secure Computer Systems: Mathematical Foundations*. MITRE, 1973. [↩](#)
- 2 Denning, D. E. *A Lattice Model of Secure Information Flow*. CACM, 1976. [↩](#)
- 3 Efstathopoulos et al. *Labels and Event Processes in the Asbestos Operating System*. SOSP, 2005. [↩](#)
- 4 Zeldovich et al. *Making Information Flow Explicit in HiStar*. OSDI, 2006. [↩](#)
- 5 Krohn et al. *Information Flow Control for Standard OS Abstractions*. SOSP, 2007. [↩](#)
- 6 Myers, A. C. *JFlow: Practical Mostly-Static Information Flow Control*. POPL, 1999. [↩](#)
- 7 Schultz, D., Liskov, B. *IFDB: Decentralized Information Flow Control for Databases*. EuroSys, 2013. [↩](#)
- 8 Pang et al. *Zanzibar: Google's Consistent, Global Authorization System*. USENIX ATC, 2019. [↩](#)
- 9 Amazon Web Services. *The Cedar Policy Language*. 2023. [↩](#)
- 10 OpenFGA project. *OpenFGA: A High-Performance and Flexible Authorization/Permission Engine*. 2023. [↩](#)
- 11 OWASP Foundation. *Top 10 for Large Language Model Applications*. 2024. [↩](#)
- 12 NIST. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. 2023. [↩](#)

